

《数据结构》第一次实验报告

雷天乐 2025110528 25 数据科学 1 班

1 实验 1

实验任务：C 语言传值调用、传地址调用

实验时间：2026/09/07

知识点：C 语言中函数参数分为传值调用与传地址调用。传值调用是将实参的副本赋值给形参，形参与实参占用不同的内存空间，函数内部对形参的修改不会改变外部实参的值。传地址调用通过指针将实参的内存地址传递给形参，形参指针指向实参所在的内存单元，函数内部通过解引用操作可以直接修改原始实参的值。

源代码

1. 值传递

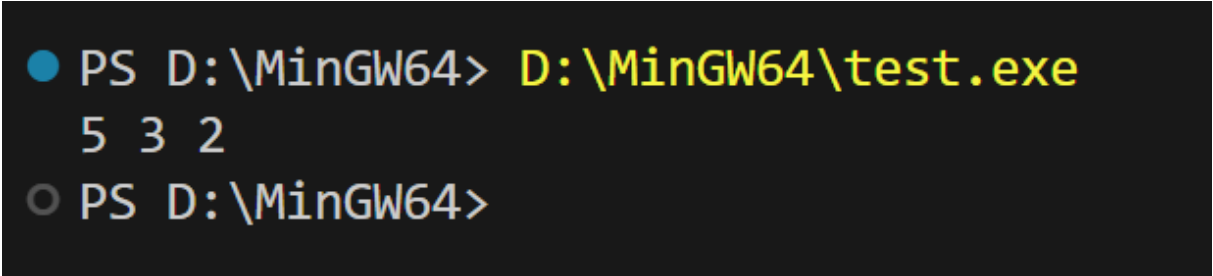
```
#include<stdio.h>
#include<stdlib.h>

void swap(int x, int y){
    int t;
    t = x; x = y; y = t;
}

int main(){
    int a,b,c;
    a = 5; b = 3; c = 2;
    if (a>b)
        swap(a,b); /*函数调用*/
    if (a>c)
        swap(a,c);
    if (b>c)
        swap(b,c);
    printf("%d %d %d",a,b,c);
```

```
    return 0;  
}
```

值传递的输出结果:

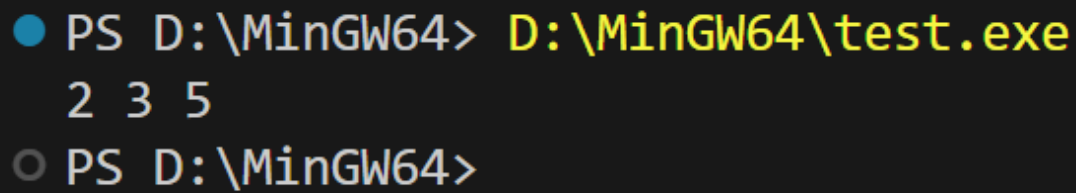


```
● PS D:\MinGW64> D:\MinGW64\test.exe  
5 3 2  
○ PS D:\MinGW64>
```

2. 地址传递

```
#include<stdio.h>  
#include<stdlib.h>  
  
void swap(int *x,int *y){  
    int t;  
    t = *x; *x = *y; *y = t;  
}  
  
int main( ){  
    int a,b,c;  
    a = 5; b = 3; c = 2;  
    if (a>b)  
        swap(&a,&b); /*函数调用*/  
    if (a>c)  
        swap(&a,&c);  
    if (b>c)  
        swap(&b,&c);  
    printf("%d %d %d",a,b,c);  
    return 0;  
}
```

地址传递的输出结果:



```
● PS D:\MinGW64> D:\MinGW64\test.exe  
2 3 5  
○ PS D:\MinGW64>
```

实验总结

本次实验通过值传递方式实现交换函数，发现函数内部修改形参并不会改变主函数中实参变量的值，由此理解了传值调用的原理：形参是实参的副本，二者拥有独立内存空间。该程序试图利用值传递完成三个数的排序，但由于值传递无法修改原始变量，程序不能实现预期排序效果。通过本次实验对比，加深了对 C 语言函数参数传递机制的认识，明白若要在函数中修改外部变量，需要使用传地址调用的方式。

2 实验 2

实验任务：C 语言的标准内存分配函数（以 realloc 为例探究）

实验时间：2026/09/07 晚 ~ 2026/09/08

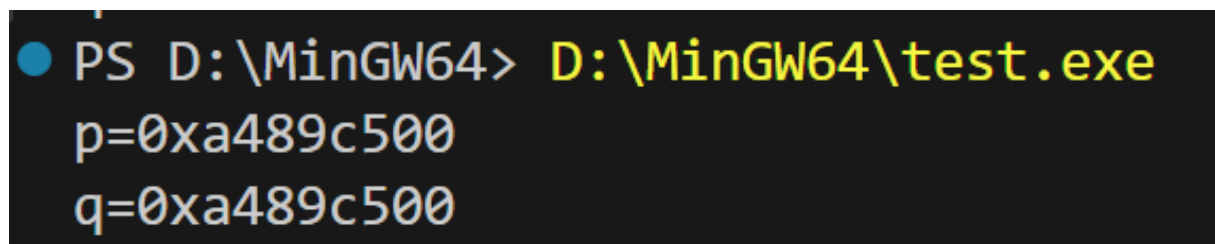
知识点：C 语言提供了 malloc、calloc、realloc 和 free 等标准内存分配函数，用于动态管理内存空间。其中，malloc 用于申请指定大小的连续内存空间，calloc 用于申请 n 个大小为 size 的连续空间，并将其初始化为零；realloc 用于调整已经分配的内存空间大小；free 用于释放之前动态申请的内存。通过这些函数，可以根据程序运行的实际需求灵活分配和释放内存，提高内存使用效率。

源代码

源代码 1: realloc 地址不变

```
#include <stdio.h>
#include <stdlib.h>
int main(){
    int *p,*q;
    p=(int*)malloc(2*sizeof(int));
    q= p;
    p=(int*)realloc(p,2*sizeof(int));
    printf("p=0x%x\n", p);
    printf("q=0x%x\n", q);
    return 0;
}
```

输出结果：



```
PS D:\MinGW64> D:\MinGW64\test.exe
p=0xa489c500
q=0xa489c500
```

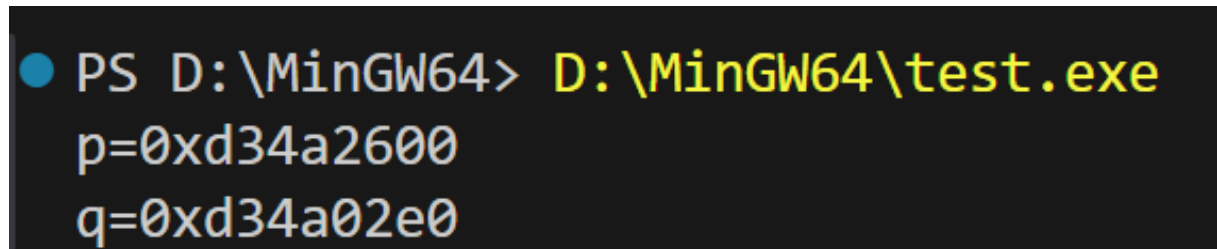
源代码 2: realloc 地址改变

```
#include <stdio.h>
#include <stdlib.h>

int main(){
```

```
int *p,*q;  
p = (int*)malloc(10*sizeof(int));  
q = p;  
p = (int*)realloc(p,100*sizeof(int));  
printf("p=0x%x\n",p);  
printf("q=0x%x\n",q);  
return 0;  
}
```

输出结果:



```
● PS D:\MinGW64> D:\MinGW64\test.exe  
p=0xd34a2600  
q=0xd34a02e0
```

实验总结

realloc 并不保证调整后的内存空间和原来的内存空间保持同一内存地址。相反, realloc 返回的指针很可能指向一个新的地址(返回同一个地址的情况是我询问老师之后调出来的)。因此,使用 realloc 后就应该改用 realloc 返回的新指针。

3 实验 3

实验任务：复数 ADT 的实现：包含 Complex.h, Complex.c, Main.c

实验时间：2026/09/08

知识点：在 C 语言中，可以使用结构体（Struct）来存放一组不同类型的数据。可以通过结构体保存复数 Z。以及要注意 Complex.h 头文件的编写中需要定义 Complex 这个结构体数据类型以及声明必要的函数，并在 Complex.c 中实现这些函数。

源代码

Complex.h 的源代码

```
#ifndef COMPLEX_H
#define COMPLEX_H

#include <stdio.h>
typedef struct{
    float realpart;
    float imagpart;
}Complex;

void Assign(Complex *Z, float Realval, float Imagval); //构造复数Z
float GetReal(Complex *Z); //返回复数Z的实部值
float GetImag(Complex *Z); //返回复数Z的虚部值
void Add(Complex z1,Complex z2,Complex *sum); //返回加和结果sum
#endif
```

Complex.c 的源代码

```
#include "complex.h"

void Assign(Complex *Z, float Realval, float Imagval){
    Z->imagpart = Imagval;
    Z->realpart = Realval;
}

float GetReal(Complex *Z){
    return Z->realpart;
}
```

```
float GetImag(Complex *Z){
    return Z->imagpart;
}

void Add(Complex z1,Complex z2,Complex *sum){
    sum->realpart = z1.realpart + z2.realpart;
    sum->imagpart = z1.imagpart + z2.imagpart;
}
```

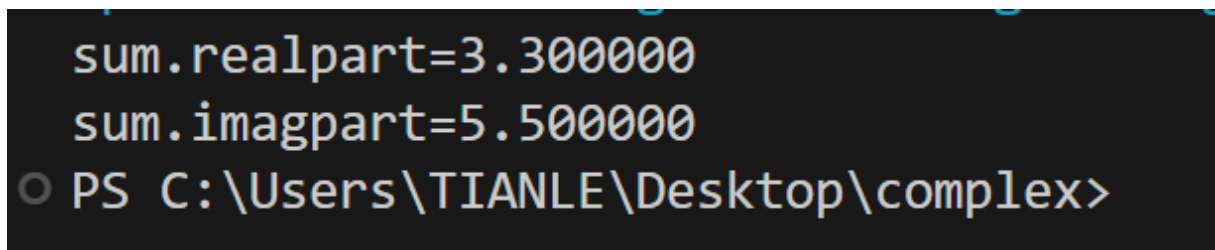
Main.c 的源代码

```
#include <stdio.h>
#include "Complex.h"
int main(){
    Complex com_1,com_2,sum;

    Assign(&com_1, 1.2, 2.3);
    Assign(&com_2, 2.1, 3.2);
    Add(com_1,com_2,&sum);
    printf("sum.realpart=%f\nsum.imagpart=%f\n",sum.realpart,sum.
        imagpart) ;
    return 0;
}
```

//这里技术上不需要再写一次stdio.h，但写上可能可维护性更好

输出结果:



```
sum.realpart=3.300000
sum.imagpart=5.500000
PS C:\Users\TIANLE\Desktop\complex>
```

实验总结

本次实验通过 Complex.h、Complex.c、Main.c 三个文件实现了复数 ADT, 掌握了用结构体封装复数数据、以及接口声明与函数实现分离的模块化编程思想。调试过程中

遇到过因编译任务未包含所有源文件而导致链接报错的问题, 通过修改 `tasks.json` 得以解决, 加深了对多文件编译链接过程的理解。最终程序正确计算出复数之和, 验证了各函数实现的正确性, 也巩固了头文件包含等基础知识点。

4 实验 4

实验任务：思考算法的有穷性：HailstoneSeq

实验时间：2026/09/08

知识点：算法的定义要求满足“有穷性”，即算法必须在有限步骤内结束。本题通过实现 Hailstone 序列（考拉兹猜想，对偶数除以 2，对于奇数乘 3 再加 1 一直到数变为 1 为止），体会“程序目前观测上会停止”与“数学上被证明一定会停止”之间的区别，理解算法有穷性并非可以想当然地凭经验假设，而应当能够被严格证明。

源代码：以输入值为 27 为例

```
#include <stdio.h>
#include <stdlib.h>

int HailstoneSeq(int n){
    int length = 1;
    while(n>1){
        printf("%d, ",n);
        (n%2==0) ? (n/=2) : (n=n*3+1);
        ++length;
    }
    if(n<=1)
        printf("%d, ",1);
    return length;
}

int main(){
    int a;
    a = HailstoneSeq(27);
    printf("length:%d",a);
    return 0;
}
```

输出结果：

```
● PS D:\MinGW64> D:\MinGW64\test.exe
27, 82, 41, 124, 62, 31, 94, 47, 142, 71, 214, 107, 322, 161, 484, 242, 121, 364, 182, 91, 274, 137, 4
12, 206, 103, 310, 155, 466, 233, 700, 350, 175, 526, 263, 790, 395, 1186, 593, 1780, 890, 445, 1336,
668, 334, 167, 502, 251, 754, 377, 1132, 566, 283, 850, 425, 1276, 638, 319, 958, 479, 1438, 719, 2158
, 1079, 3238, 1619, 4858, 2429, 7288, 3644, 1822, 911, 2734, 1367, 4102, 2051, 6154, 3077, 9232, 4616,
2308, 1154, 577, 1732, 866, 433, 1300, 650, 325, 976, 488, 244, 122, 61, 184, 92, 46, 23, 70, 35, 106
, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1, length:112
○ PS D:\MinGW64>
```

(以逗号作为数据间隔)

实验总结

数学上我们还没有证实或者证伪 Hailstone 序列一定会出现数字 1，除去本次实验，其实人们验证了很多很大的数都会出现数字 1，但由于考拉兹猜想并未被证实，我们不能仅凭该程序最后会终止于数字 1（即所谓的“有穷”）就断定其是否具有“有穷性”，算法有穷性并非可以想当然地凭经验假设，而应当能够被严格证明。

5 实验 5

实验任务：记录程序运行时间

实验时间：2026/09/09

知识点：通过 time.h 头文件统计程序 CPU 运行时间，该函数返回 'clock_t' 类型的 CPU 时钟滴答数，得到以秒为单位的耗时。通过记录计算 $1 + 2 + 3 + \dots + 1000000000$ 的不同方法的用时，比较算法的优劣。

源代码

源代码 1：for 循环累加

```
#include <stdio.h>
#include <time.h>
void fun1(){
    long long i, sum = 0;
    long long n = 1000000000;
    for(i = 1; i<= n; i++){
        sum = sum + i;
    }
    printf("%lld\n", sum);
}

int main() {
    clock_t tic,toc;
    tic=clock();
    fun1();
    toc=clock();
    printf("elapsed time is %fs\n",(double)(toc-tic)/CLOCKS_PER_SEC);
    return 0;
}
```

输出结果：

```
● PS D:\MinGW64> D:\MinGW64\test.exe
5000000000500000000
elapsed time is 1.555000s
○ PS D:\MinGW64> █
```

源代码 2: 求和公式

```
#include <stdio.h>
#include <time.h>

void fun2(){
    long long i, sum = 0;
    long long n = 1000000000;
    sum = (1 + n) * n / 2;
    printf("%lld\n", sum);
}

int main() {
    clock_t tic, toc;
    tic = clock();
    fun2();
    toc = clock();
    printf("elapsed time is %fs\n", (double)(toc - tic) / CLOCKS_PER_SEC);
    return 0;
}
```

输出结果:

```
PS D:\MinGW64> D:\MinGW64\test.exe
● 5000000000500000000
elapsed time is 0.000000s
○ PS D:\MinGW64>
```

实验总结

本实验利用 `<time.h>` 中的 `clock()` 函数测量 CPU 运行时间，对比了 for 循环逐次累加与数学求和公式两种实现 1 到 1000000000 求和算法的执行效率，循环累加需要重复执行十亿次加法运算，CPU 消耗大运行耗时较长，而求和公式仅通过一次算术运算即可得到结果，执行速度极快；实验直观体现出算法时间复杂度对程序运行效率的影响，选用时间复杂度更低的算法可以大幅提升程序性能。

6 实验 6

实验任务：跑通对数阶 $O(\log n)$ 算法示例 countOnes

实验时间：2026/09/10

知识点：本实验通过实现 countOnes 算法统计整数二进制表示中 1 的个数，核心是按位与运算符 & (用 $1 \& n$ 提取最低位并判断是否为 1) 和右移运算符 $>>$ (用 $n >>= 1$ 每次将 n 缩减至原来的一半，从而遍历完所有二进制位)。由于循环次数取决于 n 的二进制位数而非数值大小，设循环执行 x 次后 n 缩减至 0，可推出 $n = 2^{(x-1)}$ ， $x = 1 + \log_2 n$ ，因此该算法的时间复杂度为 $T(n) = O(\log n)$ ，属于对数阶算法。

源代码

```
#include <stdio.h>

int countOnes(unsigned int n)
{
    int ones = 0;           // 计数器复位
    while (0 < n)           // 在n缩减至0之前，反复地执行
    {
        ones += (1 & n);    // 检查最低位，若为1则计数
        n >>= 1;           // 每右移一位，n都至少缩减一半
    }
    return ones;           // 返回计数
}

int main()
{
    unsigned int n;
    printf("输入n: ");
    scanf("%d",&n);
    int result = countOnes(n);
    printf("n = %u, 二进制中1的个数 = %d\n", n, result);
    return 0;
}
```

输出结果：

```
● PS D:\MinGW64> D:\MinGW64\test.exe  
  输入n: 441  
    n = 441, 二进制中1的个数 = 6  
○ PS D:\MinGW64> █
```

n=441 的情况

```
● PS D:\MinGW64> D:\MinGW64\test.exe  
  输入n: 7  
    n = 7, 二进制中1的个数 = 3  
○ PS D:\MinGW64> █
```

n=7 的情况

```
● PS D:\MinGW64> D:\MinGW64\test.exe  
  输入n: 27  
    n = 27, 二进制中1的个数 = 4  
○ PS D:\MinGW64> █
```

n=27 的情况

实验总结

本次实验实现了 countOnes 算法, 通过按位与运算 (&) 提取二进制最低位、右移运算 (>>) 逐位遍历, 统计出整数二进制表示中 1 的个数, 并验证了不同输入下算法输出的正确性。通过对算法执行次数的推导, 理解了该算法的时间复杂度为 $O(\log n)$, 体会到对数阶算法的运行时间只与输入的位数相关, 而非输入的数值大小相关, 因此在处理数值规模很大的输入时依然能保持很高的效率。本次实验也让我进一步掌握了位运算 (按位与、右移) 在实际算法设计中的应用。

7 实验 7

实验任务：比较空间复杂度

实验时间：2026/09/10

知识点：通过比较循环方法和递归方法实现 printN 函数（输入一个正整数 N 后，顺序打印从 1 到 N）输入不同的 N=10000, 1000000, 100000000 并比较输出情况和方式，以判断空间复杂度优劣。

源代码

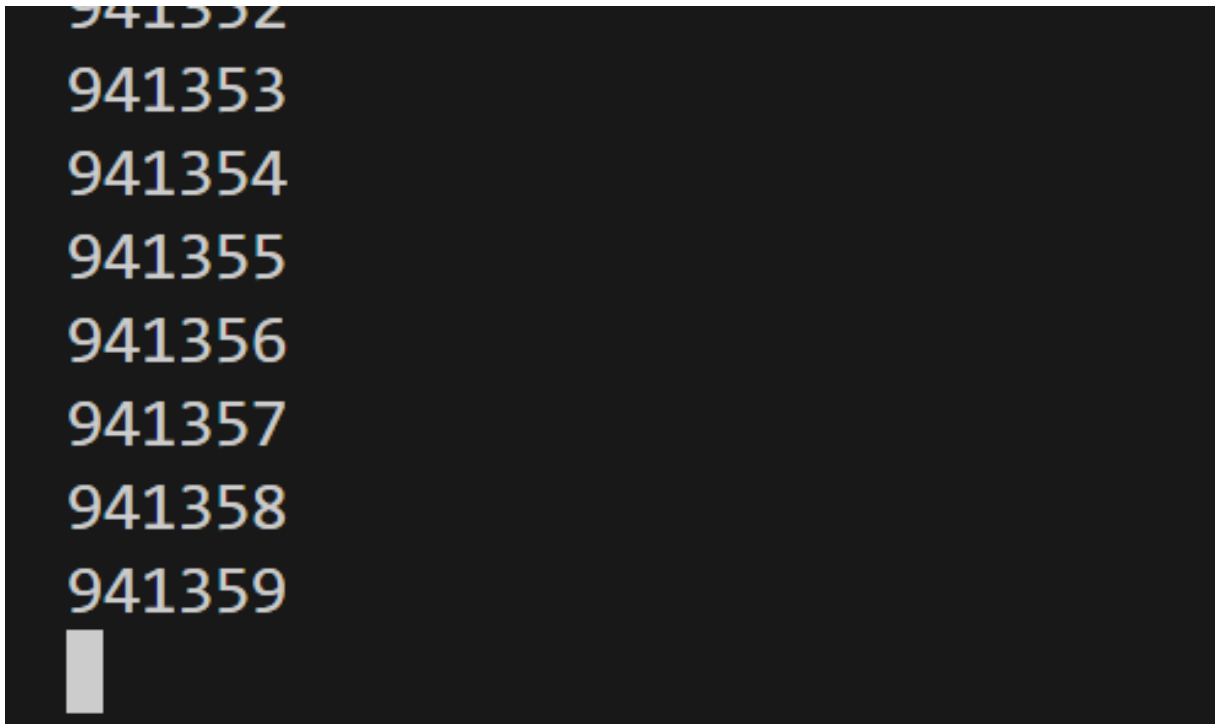
源代码 1: for 循环

```
#include <stdio.h>

int printN_01(long long N){
    int i;
    for (i=1; i<=N; i++){
        printf("%d\n",i);
    }
    return 0;
}

int main(){
    long long n ;
    printf("输入N: ");
    scanf("%lld", &n);
    printN_01(n);
    return 0;
}
```

输出结果：



(取了不同的 3 个 N 都可以流畅运行，本截图是正在输出数据的一帧)

源代码 2: 递归

```
#include <stdio.h>

int printN_02(long long N){
    if (N){
        printN_02(N-1);
        printf("%d\n",N);
    }
    return 0;
}

int main(){
    long long n ;
    printf("输入N: ");
    scanf("%lld", &n);
    printN_02(n);
    return 0;
}
```

输出结果:

```
9996
9997
9998
9999
10000
```

N=10000 正常运行

```
PS D:\MinGW64> D:\MinGW64\test.exe
输入N: 1000000
```

N=1000000 无法正常运行 (左上角红色叉号)

```
PS D:\MinGW64> D:\MinGW64\test.exe
PS D:\MinGW64> 输入N: 100000000
```

N=100000000 无法正常运行 (左侧红色叉号)

实验总结

循环版本 printN_01 无论输入 N 取较大数值, 程序均可以完整执行完毕, 不会发生崩溃, 说明它只消耗固定栈空间, 空间复杂度 $O(1)$; 递归版本 printN_02 随着 N 增大, 递归调用深度同步增加, 栈内存持续累积, 当 N 超过系统栈可承受的最大深度, 程序发生栈溢出崩溃, 体现递归算法空间复杂度为 $O(N)$ 。时间复杂度两者均为 $O(N)$, 但空间开销差异巨大。